

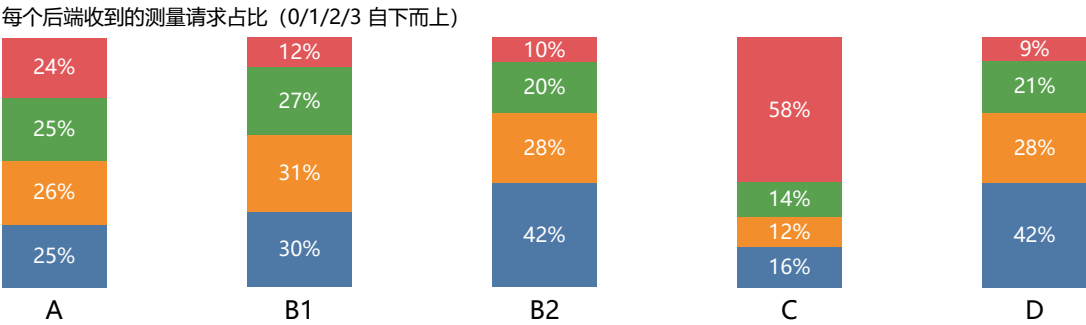
比较并改进 Ray Serve 路由 —— 挑战三实验报告

HW3-0102603133 | SGLang 0.5.14 · Ray 2.56.0 · Qwen3-0.6B | 单卡 RTX 4090D (4 后端共享) · Ubuntu 22.04 容器

1. 实验环境与固定负载

按课程要求，四组对照均通过同一 Ray Serve HTTP 入口回放官方固定负载 mooncake\_prefix\_workload\_v2\_seed2026.jsonl (SHA-256 f154ba95...75b03 校验通过：64 前缀族 / 64 预热 / 2048 测量请求，计划时长 29.54s，含 steady→burst→recovery 三阶段与一个高频热点族)。模型、SGLang 参数 (--mem-fraction-static 0.18 --attention-backend triton --disable-cuda-graph)、副本拓扑与压测端配置全程不变；每轮由 run\_round.sh 重新部署对应路由配置 → 对四个后端 POST /flush\_cache → 官方 run\_workload.py 顺序跑 64 条预热、再按到达时间回放 2048 条测量。

与作业说明的两处偏差 (如实说明)：① 环境为单张 24GB GPU (未能预约到 4 卡)，用 ray.cluster\_utils.Cluster 在同一 GPU 上建 1 head + 4 逻辑 worker，四个 SGLang 后端共享该 GPU；四组的负载、模型与后端配置完全一致，组间横向对比成立；但绝对吞吐与延迟数值不能与 4 卡环境直接比较。② 容器默认 ulimit -n=1024，官方压测端 (TCPConnector(limit=0)、2048 并发连接) 会耗尽 fd，首轮曾出现 873/2048 条 OSError(24) Too many open files，已统一 ulimit -n 65535 后重跑；组 A/C 队列排空需 ~360/450s，客户端单请求超时统一放宽为 900s (B/D 的最大延迟远低于 300s，不受影响)。五组最终均为 2048/2048 全部成功。



2. target1: 前缀缓存测量 (shared vs dispersed)

按挑战二规范执行：Radix Cache 开启，原生 /generate 接口发送 input\_ids 并流式接收；每组 32 条测量请求、最大并发 8、temperature=0、max\_new\_tokens=16、ignore\_eos=true、sampling\_seed=2026。共享前缀组先发送 1 条含共享前缀的预热请求 (不计入结果)；两组测量前均 flush\_cache 并确认成功。脚本 src/target1/measure\_prefix\_cache.py。

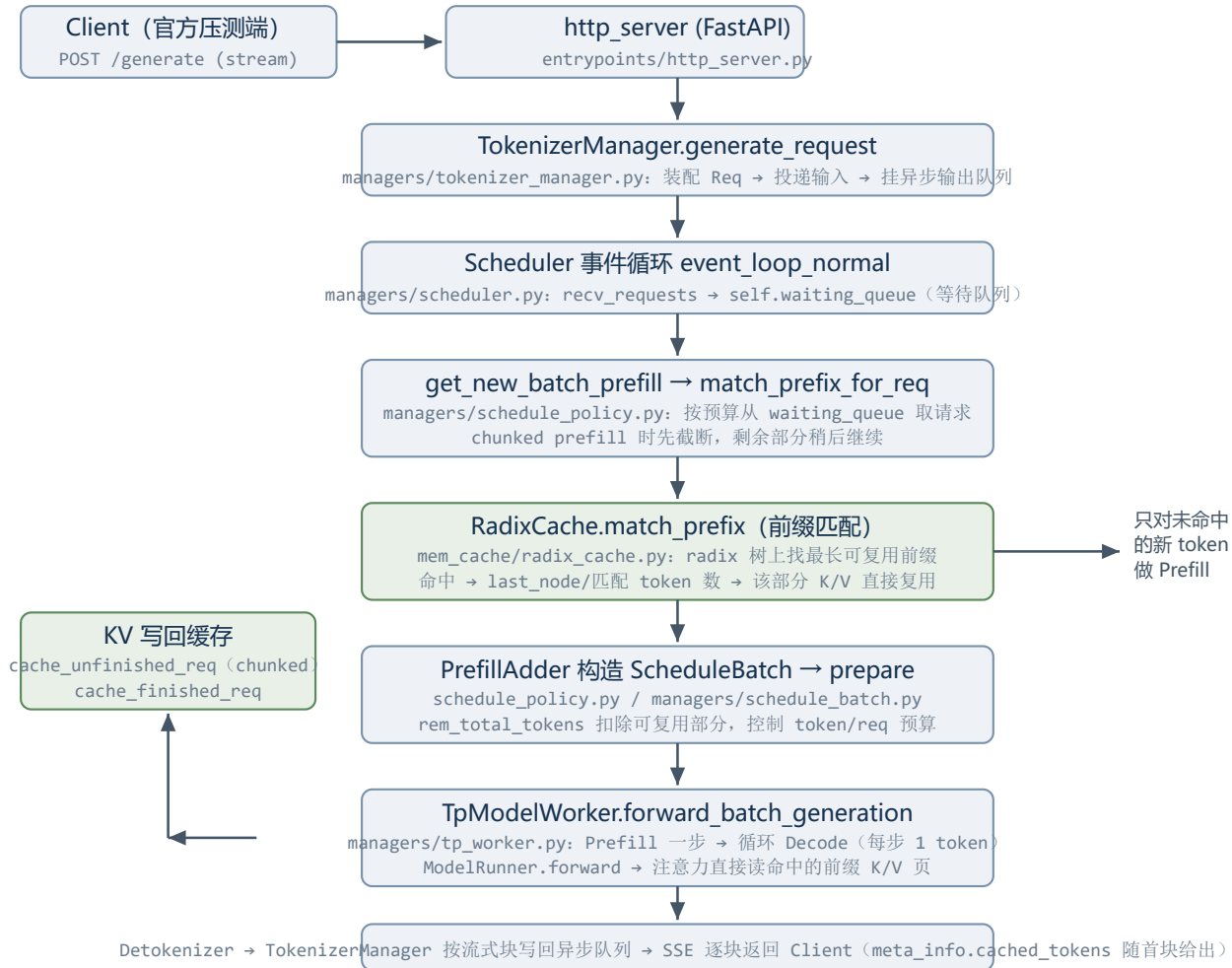
| 负载                     | 成功    | 吞吐 (req/s) | 缓存命中率  | 实际 Prefill token | TTFT p50/p95 (ms) | E2E p50/p95 (ms) |
|------------------------|-------|------------|--------|------------------|-------------------|------------------|
| 共享前缀 (2048 共享 + 64 独立) | 32/32 | 16.74      | 97.06% | 1,985            | 95.8 / 106.7      | 471 / 489        |
| 分散前缀 (2112 全不同)        | 32/32 | 11.73      | 0.00%  | 67,584           | 205.8 / 304.8     | 674 / 691        |

结论：共享前缀组命中率达 97.06% (理论上限 = 2048/2112 = 97.0%，预热请求把首条的后缀也留在了树上)，实际执行 Prefill 仅 1,985 token，是分散组 (67,584) 的 3.4%；TTFT p50 降低约 53%。TPOT 中位数从 24.7ms 升到分散组 31.7ms 的方向相反 (共享组更低)，原因是分散组每条请求都要做 2112 token 的 Prefill，Decode 步骤与后续请求的 Prefill 交错排队，拉大了相邻输出 token 的间隔。

从因果自注意力解释前缀复用：Attention(Q,K,V)=softmax(QK<sup>T</sup>/√d<sub>k</sub>)V。因果掩码使第 i 个 token 只依赖位置 ≤i 的 K/V (K<sub>i</sub>=h<sub>i</sub>W<sub>K</sub>, V<sub>i</sub>=h<sub>i</sub>W<sub>V</sub> 只由该 token 自身及其之前的内容决定)。因此「相同 token + 相同位置」的前缀，其 K/V 完全相同，可以整体复用；Prefill 只需计算前缀之后的新 token (共享组即 64 token 后缀)，并在注意时把 Q<sub>new</sub> 与缓存的前缀 K/V 做 attention。命中后仍需计算的是：后缀 token 的 K/V、最后位置的 Q·K<sup>T</sup> 注意力与采样。TTFT 因此大幅下降 (Prefill 量骤减)，而 Decode 阶段每步的计算量与 Prefill 无关，TPOT 基本不变——两者呈现不同变化。

### 3. 一次 /generate 请求在 SGLang v0.5.14 中的流程

流程图与说明合计不超过 2 页；关键函数均已在服务器安装的 v0.5.14 源码 ( .../site-packages/sglang/srt/ ) 中逐一核对。



**请求如何进入等待队列：**HTTP 层收到 /generate 后交给 TokenizerManager.generate\_request，它构造 Req 并经 scheduler 的输入流异步发给 Scheduler 进程；event\_loop\_normal 每轮 recv\_requests 后将新请求 append 进 self.waiting\_queue，等待每轮调度。

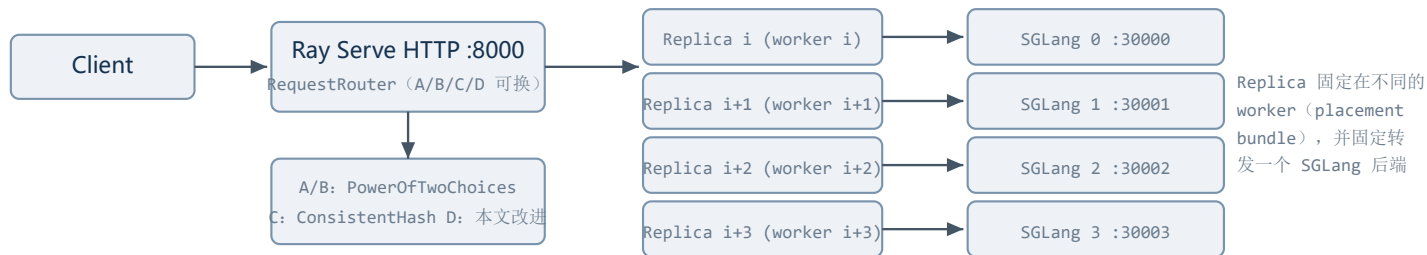
**前缀匹配如何减少本轮 Prefill token：**get\_new\_batch\_prefill 对每个候选请求调用 match\_prefix\_for\_req → RadixCache.match\_prefix，在 radix 树上找最长匹配；匹配到的 token 不再进入 Prefill 批次（PrefillAdder 按 rem\_total\_tokens 扣除后只排新 token），于是 prompt\_len - matched\_len 才是实际 Prefill 量。

**新产生的 KV 如何写回缓存：**chunked prefill 的中间态由 cache\_unfinished\_req 把已计算部分插入 radix 树；请求完成后由 cache\_finished\_req 将完整序列的 KV 页接入树中供后续请求复用，并在显存紧张时按 LRU 淘汰。

**生成结果如何流式返回：**每个 Decode 步产出一个 token，经 Detokenizer 解码后发回 TokenizerManager；generate\_request 的异步生成器把增量文本以 SSE data: 块写回 HTTP 连接，末块携带 meta\_info（含 cached\_tokens），客户端据此统计命中率。

| 环节                | 关键函数 / 类                                     | 文件 (sglang/srt/ 下)                                   |
|-------------------|----------------------------------------------|------------------------------------------------------|
| HTTP 接收           | /generate 路由                                 | entrypoints/http_server.py                           |
| 装配与流式回调           | TokenizerManager.generate_request            | managers/tokenizer_manager.py                        |
| 事件循环 / 等待队列       | Scheduler.event_loop_normal、waiting_queue    | managers/scheduler.py                                |
| Prefill 调度        | get_new_batch_prefill、PrefillAdder           | managers/scheduler.py、managers/schedule_policy.py    |
| 前缀匹配              | match_prefix_for_req、RadixCache.match_prefix | managers/schedule_policy.py、mem_cache/radix_cache.py |
| 批次结构              | ScheduleBatch / ModelWorkerBatch             | managers/schedule_batch.py                           |
| 执行 Prefill/Decode | TpModelWorker.forward_batch_generation       | managers/tp_worker.py                                |
| KV 写回             | cache_finished_req / cache_unfinished_req    | mem_cache/radix_cache.py                             |
| 流式输出              | Detokenizer → TokenizerManager 流式队列          | managers/detokenizer_manager.py、tokenizer_manager.py |

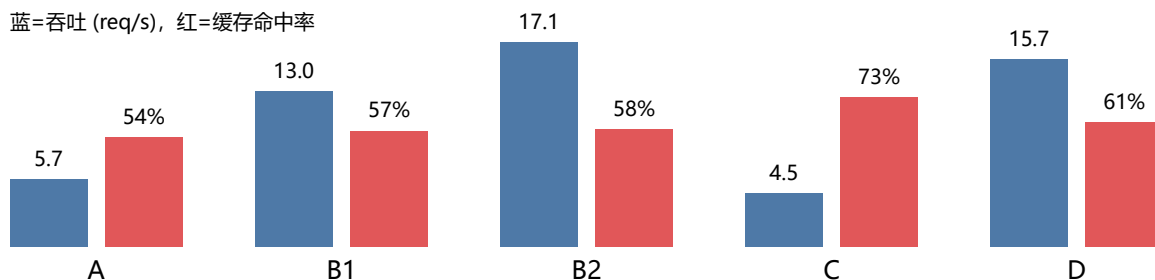
## 4. target3: 四组路由对照



### 4.1 主表 (每组 2048 条测量请求, 全部成功)

| 组  | 路由              | max_ongoing | 成功率  | 吞吐 (req/s) | 缓存命中率 | 实际 Prefill | TTFT p95 (s) | E2E p95 (s) | 后端分布 0/1/2/3     |
|----|-----------------|-------------|------|------------|-------|------------|--------------|-------------|------------------|
| A  | p2c             | 5           | 100% | 5.67       | 53.8% | 1,494k     | 312.29       | 315.88      | 512/528/516/492  |
| B1 | p2c             | 16          | 100% | 13.04      | 56.7% | 1,399k     | 113.83       | 116.99      | 622/627/557/242  |
| B2 | p2c             | 64          | 100% | 17.13      | 57.7% | 1,368k     | 69.80        | 73.78       | 853/580/414/201  |
| C  | consistent-hash | 64          | 100% | 4.51       | 73.4% | 861k       | 386.77       | 390.74      | 330/249/284/1185 |
| D  | affinity+负载感知   | 64          | 100% | 15.70      | 60.8% | 1,266k     | 74.38        | 78.40       | 854/580/422/192  |

负载计划 29.54s 内发完全部请求 (~69 req/s), 远超单卡 4 后端的服务能力, 因此各组都在过饱和状态下排队; 这放大了组间差异, 也使「负载过高时路由策略如何行为」成为可观察对象。因此主表中的 p95 应读作过饱和状态下的队列排空时间, 不代表服务的尾延迟能力: 组 A 的 E2E p95 315.9s 中, TTFT p95 已占 312.3s, 首字之后的实际生成只有 3.6s; 组 C 对应为 390.7s、386.8s 与 4.0s。这组数字是单卡 4 后端过饱和的结果, 不宜与其他 4 卡环境的队伍横向比较。下图为吞吐与命中率对照:



### 4.2 组 B: max\_ongoing\_requests 的两个候选

16 → 64 同时改善吞吐 (13.04 → 17.13 req/s) 与延迟 (TTFT p95 113.8s → 69.8s), 故 **B\_PICK = 64**。解释: 本环境请求在 Serve 代理排队而非后端排队时, GPU 利用率不足; 放宽 max\_ongoing\_requests 让每个 SGLang 后端能组出更大的 Prefill/Decode batch, 摊薄了单请求开销。默认值 A (=5) 过小: 4 副本同时至多 20 条在后端, 361s 才排空队列, 吞吐仅 5.67 req/s。

### 4.3 组 C: 前缀亲和的收益与热点

一致性哈希把同一 X-Session-Id (= 前缀族) 恒定路由到同一副本: 缓存命中率升至 73.4% (B2 为 57.7%), 实际 Prefill token 从 1,368k 降到 861k (-37%), 验证了「前缀亲和 → 缓存收益」。但代价同样显著: 高频热点族 (768 条 superhot 请求) 被钉在单一后端, 该后端收到 57.9% 的请求 (1185/2048), 吞吐崩至 4.51 req/s、TTFT p95 达 386.8s。热点族的 TTFT p50 = 208s, 而 cold 族仅 5.7s——亲和把缓存收益变成了排队代价。

| 热度层              | B2 命中率 / TTFT p50 | C 命中率 / TTFT p50 | D 命中率 / TTFT p50 |
|------------------|-------------------|------------------|------------------|
| cold (n=240)     | 14.9% / 33.00s    | 46.1% / 5.69s    | 20.5% / 36.00s   |
| warm (n=480)     | 21.6% / 32.54s    | 56.1% / 6.89s    | 27.9% / 32.58s   |
| hot (n=560)      | 54.5% / 33.07s    | 68.9% / 12.93s   | 58.9% / 34.53s   |
| superhot (n=768) | 95.2% / 33.61s    | 95.4% / 207.99s  | 94.7% / 28.72s   |

按前缀族热度分层 (B2/C/D): C 让 cold/warm/hot 层全面受益 (命中率 +25~34pt、TTFT 大幅下降), 但 superhot 层 TTFT p50 从 33.6s 恶化到 208.0s。

### 4.4 组 D: 亲和 + 负载感知回退

AffinityLoadAwareRouter (src/target3/routers.py, 继承 ConsistentHashRouter) 只用两类信号: **当前请求的 X-Session-Id** (一致性哈希给出亲和副本) 与 **当前服务状态** (经 \_probe\_queue\_lens 实时探测的各副本排队长度, 与 p2c 同一机制)。规

则：亲和副本的在队长度  $\geq 75\% \times \text{max\_ongoing}$ （默认）时，改排当前最空闲副本，并把 每次回退写入 `router_fallbacks.jsonl`。不读任何未来请求。

**效果：**命中率 60.8%、Prefill 1,266k token，在 B2 与 C 之间保住了约一半的缓存收益；吞吐 15.70 req/s、TTFT p95 74.4s，基本恢复到 B2 水平（17.13 / 69.8s），最最后端的请求占比（四个后端中承载请求数最多者的 计数  $\div$  2048，与 4.1 主表同口径）从 57.9% 降到 41.7%。本轮共记录 47,664 次回退决策、覆盖 64/64 个前缀族——决策数远大于请求数，是因为过饱和和下一请求在被亲和副本拒绝排队期间会被路由器反复重估；superhot 族（768 条）占回退的三分之一以上。

**代价：**① 每次路由决策多一次队列长度探测（缓存未命中时为 RPC），路由层 开销增加；② 回退破坏部分亲和，命中率（60.8%）明显低于 C（73.4%）；③ 与 B2 相比 吞吐 -8.3%、E2E p95 +6.3%——在本环境（后端总容量 < 到达率）下，p2c 的 "选两个比负载"已能避开热点，D 的边际收益主要体现在省 Prefill（1266k vs 1368k token，-7.4%）与更高的命中率上；若后端 KV 池更紧张或热点更多，该收益会被放大。

4.5 分阶段行为与压测端健康检查

| 阶段               | A 命中率 / TTFT p50 | B2 命中率 / TTFT p50 | D 命中率 / TTFT p50 |
|------------------|------------------|-------------------|------------------|
| steady (n=512)   | 50.1% / 36.98s   | 56.8% / 3.56s     | 65.7% / 2.54s    |
| burst (n=1024)   | 56.0% / 161.59s  | 57.8% / 33.32s    | 59.0% / 43.01s   |
| recovery (n=512) | 53.2% / 281.65s  | 58.5% / 60.60s    | 59.7% / 42.56s   |

steady 阶段（前 512 条）TTFT 仅数秒：缓存已被预热填好；burst 阶段 1024 条集中到达，把队列推深，TTFT 跳升一个量级；recovery 阶段到达率下降，命中率与 TTFT 逐步回落。client\_queue p95（A 0.6ms / B1 0.7ms / B2 0.6ms / C 0.6ms / D 0.6ms）与 dispatch\_lag p95（3.2ms / 3.8ms / 4.8ms / 5.6ms / 8.7ms）均在毫秒级，压测端不是瓶颈，瓶颈在路由与后端。

## 5. 结论与复现

① 前缀缓存的收益可以被路由策略"兑现"或"浪费": C 组证明了亲和的最大缓存收益 (−37% Prefill), 也暴露了其热点代价;  
② 负载感知回退 (D) 用少量命中率换回了吞吐与 尾部延迟, 是工程上更稳的折中; ③ `max_ongoing_requests` 是单卡多后端场景下最敏感的旋钮 (A→B2 吞吐 3 倍)。

复现: `cd src/target3 && NUM_GPUS=1 bash launch_sglang.sh && for g in A_default B_cand1 B_cand2 C_affinity D_improved; do bash run_round.sh $g; done` (`B_PICK=64 TIMEOUT_S=900` 已固化为 脚本默认/说明); target1: `python src/target1/measure_prefix_cache.py --base-url http://127.0.0.1:30000 --output-dir results/target1`。结果目录与报告表格对应关系见 README.md 第 6 节。

## 6. 作业感受

**(1) 如何完成本次挑战:** 按"环境→数据→分析"的顺序推进: 先在租用的 GPU 容器里装好 `sglang-env` / `ray-env` 双环境并下载 Qwen3-0.6B; 再写 `serve_app.py` (Cluster 逻辑节点 + placement group 固定副本到 worker) 与 `run_round.sh` (每轮重部署→flush→官方压测→校验); AI 全程协助搭建、排障与写作, 关键排障均以数据为准 (如发现 873 条失败的根因是 fd 上限而非服务能力, 用 `/proc/<pid>/limits` 验证后修复重跑; A/C 组超时由 300s 客户端 预算导致, 通过队列排空时间推算后统一放宽并整链重跑)。

**(2) 最困难的部分:** 最折腾的是排查压测失败。第一轮 A 组 873 条请求直接连不上, 一开始以为是四个后端扛不住 69 req/s, 或者端口哪里配错了, 翻了半天日志 没结论, 最后是在 `requests.csv` 的 error 列里看到完整报错——`Too many open files`: 是压测端自己把文件描述符用光了 (容器默认 `ulimit -n 1024`, 官方压测脚本开 2048 个连接且不设上限)。改完这个, A 和 C 又各有上百条超时, 这次算了下队列排空时间才发现是压测端 300 秒的单请求 预算不够用, 服务本身没问题。这两次都差点把方向带偏, 后来养成一个习惯: 每个结论都回到原始 CSV/JSON 里对一遍数字再下, 不在汇总指标上想当然。

**(3) 对今后科研工作的启发:** 这次算是完整走了一遍"现象 → 数据 → 根因 → 可复现"的流程, 回头看, 从 0 到 1 的工作至少要包含这么几块: 先把环境搭到能稳定 复现并把每轮实验脚本化, 不然五组对照根本保证不了同口径; 其次是测量本身要当成 实验对象来对待——这次压测端 fd 和超时这两个"客户端配置"就伪装成了"服务端不行"; 再次是如实记录偏差, 单卡替代四卡、超时放宽这些写进 README 比藏着好, 审的人自己 会判断。对我之后的科研, 最大的启发是: 实验框架的可靠性本身就是工作量, 而且应该 是最先做的那一部分, 否则后面所有结论都悬着。